

Cox-Ingersoll-Ross (CIR) Process Age Inference Algorithm for Age Estimation Manual (v.1.0)

Taehee Lee¹, Lorraine E. Lisiecki², Samuel R. Newall², Yuxin Zhou³, and Charles E. Lawrence⁴

¹Department of Earth, Environmental & Planetary Sciences, Brown University, Providence, RI, USA

²Department of Earth Science, University of California Santa Barbara, Santa Barbara, CA, USA

³School of Earth and Atmospheric Sciences, Georgia Institute of Technology, Atlanta, GA, USA

⁴Division of Applied Mathematics, Brown University, Providence, RI, USA

Table of Contents

1. Introduction
2. Installation
3. Preparing input files
4. Running the algorithm
5. Output files

1. Introduction

The Python software for the Cox-Ingersoll-Ross (CIR) Process Age Inference Algorithm is designed to construct radiocarbon age models, benthic $\delta^{18}\text{O}$ -aligned age models, and multiproxy age-depth models (age models that leverage age information from both dating techniques). We regard this software as the first half of the next version of BIGMACS [Lee and Rand et al. (2023), “Bayesian age models and stacks: combining age inferences from radiocarbon and benthic $\delta^{18}\text{O}$ stratigraphic alignment”, *Climate of the Past*]; the second half will be about the stack construction, which will be available soon.

Here we provide instructions to download the software, prepare the required input files, construct age-depth models, and analyze the results. It is our hope that this software provides its users with a less subjective and more realistic method to construct ocean sediment core age models.

When you are using this software, please cite the following paper:

Lee et al. (2026), “Scalable and consistent sedimentation rate modeling via the Cox-Ingersoll-Ross (CIR) process for ocean sediment core age model development”, in review and submitted to *Geoscientific Model Development*.

Happy chronology building!

2. Installation

The open-sourced software package can be installed from GitHub using the following link:

https://github.com/eilion/CIR_Process_Aligner

The software was developed and tested on Python 3.12.2 and requires the following libraries:

NumPy; pandas; Matplotlib; SciPy; and pathlib

An example run is included with the installation. To confirm that the software is running properly, open and execute cell of the Jupyter Notebook `‘/main.ipynb’`. This will begin a multiproxy alignment for the example core MD95-2042 (up to 18 meters) to the DNEA stack [Lee and Rand et al., 2023]. Also included are the input and output folders for the cores GeoB9508-5, GIK13289-2, MD99-2334, MD95-2042 (up to 17.2 meters), and ODP658C.

3. Preparing input files

To construct age-depth models, create a new folder in `/Inputs` with the desired run name. All proxy data files should be of the csv format. See one of the input folders as an example for structure and formatting.

3.1. data_14C.csv stores the radiocarbon and calibration data for the sediment core and has the following six columns: (1) depth (in meters); (2) age (in kiloyears); (3) error (in kiloyears); (4) dR (in kiloyears); (5) dSTD (in kiloyears); and (6) cc (`‘1’` for IntCal20, `‘2’` for Marine20, and `‘3’` for SHCal20).

3.2. data_d18O.csv stores the $\delta^{18}\text{O}$ data for the sediment core and has two columns: (1) depth (in meters); and (2) $\delta^{18}\text{O}$ (in permils).

3.3. data_additional_age_info.csv stores any additional age information (e.g., tephra layers, tie points, magnetic reversals, etc.) and has three columns: (1) depth (in meters); (2) age (in kiloyears); and (3) unct (in kiloyears). Age information is fed as a Gaussian distribution with mean `‘age’` and standard deviation `‘unct’`.

If a core does not have a certain proxy type, you do not need to put the corresponding csv file in the input folder. To specify the stack to be aligned, see `Stacks/DNEA.csv` as an example. It consists of three columns: (1) age (in kiloyears); (2) mean (in kiloyears); and (3) sigma (in kiloyears). The format corresponds to stack files in BIGMACS.

4. Running the algorithm

To run the algorithm, just follow the steps 1 to 5 specified in the Jupyter Notebook `‘/main.ipynb’`. Users can find how to configure the settings there, with detailed descriptions. A code for visualization is suggested in the step 6. The folder `/C14_individual` is unnecessary for running the algorithm but for the visualization.

5. Output files

All output files will be saved in txt and csv files.

5.1. parameters.txt stores the given hyperparameters and estimated parameters.

5.2. age_summary.csv stores some estimates from the age-depth model samples at the depths of proxy observations.

5.3. age_samples.txt stores the sampled age-depth models at the depths of proxy observations, one by each column.

5.4. age_inducing_summary.csv stores some estimates from the age-depth model sampled at the inducing depths.

5.5. age_inducing_samples.txt stores the sampled age-depth models at the inducing depths, one by each column.

5.6. acc_rate_inducing.txt stores the sampled sedimentation rates at the depths in **acc_rate_inducing_depths.txt**.

5.7. _14C_summary.csv stores the input radiocarbon data and their age estimates.

5.8. d18O_summary.csv stores the input $\delta^{18}\text{O}$ data and their age estimates.

5.9. ETC_summary.csv stores the input additional age information and their age estimates.

In addition, the step 6 of the Jupyter Notebook ‘/main.ipynb’ will return an image file ‘figure.png’ together with the above output files.